
PORTAFOLIO ACTUARIAL

Proyecto 05 • SQL + Positron

Reconciliación Transaccional de Datos Masivos y Cálculo de Prima Devengada

Window Functions • Pro Rata Temporis • Optimización DQL

Área: Análisis Actuarial • Administración de Riesgos

Herramientas: PostgreSQL • DDL/DML/DQL

Modelo: Devengamiento Pro Rata Temporis • Consistencia Referencial

Ramo: Seguros Diversos / Multirramo

Índice

1	Introducción al Proyecto	3
2	Marco Teórico y Matemático	4
2.1	Cálculo de Prima Devengada (Pro Rata Temporis)	4
2.2	Consistencia Referencial de Siniestros	4
3	Arquitectura del Dataset Sintético	5
4	Pipeline de Reconciliación SQL	6
4.1	Paso 1: Generación del Entorno y Estadísticas	6
4.2	Paso 2: CTEs de Estado Histórico	6
4.3	Paso 3: Evaluación Analítica mediante Window Functions	6
4.4	Paso 4: Cruce y Consolidación Actuarial	7
5	Métricas de Optimización Computacional	8
6	Referencias	9

1. Introducción al Proyecto

En el sector asegurador, el analista actuarial invierte una porción significativa de su tiempo operativo en la extracción, limpieza y cruce de datos masivos transaccionales. Las evaluaciones técnicas en compañías multinacionales exigen un dominio riguroso de motores de bases de datos relacionales para cuadrar la información financiera y validar la coherencia técnica de las carteras.

Este proyecto implementa un entorno analítico en **PostgreSQL** utilizando **Positron IDE**. Abarca la generación sintética de un millón de pólizas, la configuración de índices de búsqueda optimizados (B-Tree), y el uso de *Common Table Expressions* (CTEs) y *Window Functions* para calcular la prima devengada histórica y reconciliar el escurrimiento de siniestros, mitigando la complejidad computacional en carteras de alto volumen.

2. Marco Teórico y Matemático

2.1 Cálculo de Prima Devengada (*Pro Rata Temporis*)

El objetivo primario de la reconciliación es determinar el monto exacto de prima que la aseguradora ha ganado legalmente por la asunción del riesgo hasta una fecha de corte t . Bajo el método de base diaria (*Pro Rata Temporis*), la prima devengada para una póliza i , sujeta a un histórico de m alteraciones o endosos, se define como:

$$Prima_Devengada_i(t) = \sum_{k=1}^m P_{i,k} \times \left(\frac{\max(0, \min(t, F_fin_{i,k}) - F_inicio_{i,k})}{F_fin_{i,k} - F_inicio_{i,k}} \right)$$

Donde $P_{i,k}$ representa la prima transaccional vigente durante el intervalo k , mientras que $F_inicio_{i,k}$ y $F_fin_{i,k}$ corresponden a las fechas efectivas de dicho intervalo contractual.

2.2 Consistencia Referencial de Siniestros

Para cuantificar el monto real de pérdidas incurridas, se debe garantizar que el cruce lógico desestime reclamaciones improcedentes. Para todo siniestro S_j asociado a la póliza i , su fecha de ocurrencia $T(S_j)$ debe verificar la condición estricta de vigencia:

$$F_emision_i \leq T(S_j) \leq F_cancelacion_i$$

Cualquier siniestro fuera de esta desigualdad es etiquetado como extemporáneo y excluido del cálculo de la siniestralidad válida, liberando la reserva correspondiente.

3. Arquitectura del Dataset Sintético

Para emular un escenario de *Big Data*, se estructuró un pipeline en DML (Data Manipulation Language) que inyecta en el motor de base de datos un portafolio sintético masivo mediante funciones estocásticas.

Entidad	Volumetría	Comportamiento Lógico
polizas	1,000,000	Génesis mediante <code>generate_series</code> . Fechas base iteradas probabilísticamente en un ciclo bianual (2024-2025).
endosos	~ 150,000	Modificaciones asignadas al azar (<code>random() <= 0.15</code>). Contiene aumentos de suma y cancelaciones anticipadas.
siniestros	~ 200,000	Frecuencia del 20%. Introduce ruido estocástico deliberado con siniestros generados fuera de la vigencia teórica.

Nota de Optimización: El cálculo iterativo cruzado de más de un millón de registros dispara la complejidad. Para mitigar escaneos de bucle anidado ($O(n^2)$), se ejecutó el comando `ANALYZE` y se construyeron índices B-Tree en las llaves foráneas y fechas, reduciendo la búsqueda a $O(\log n)$.

4. Pipeline de Reconciliación SQL

4.1 Paso 1: Generación del Entorno y Estadísticas

Se construyen índices estructurales y se fuerza la recolección de estadísticas para garantizar que el planificador (Query Planner) elija algoritmos eficientes (*Hash Join*).

```
CREATE INDEX idx_endosos_poliza ON endosos(id_poliza);
CREATE INDEX idx_siniestros_poliza ON siniestros(id_poliza);
CREATE INDEX idx_siniestros_fecha ON siniestros(fecha_ocurrencia);

ANALYZE polizas;
ANALYZE endosos;
ANALYZE siniestros;
```

4.2 Paso 2: CTEs de Estado Histórico

Se definen bloques lógicos aislados (*Common Table Expressions*) para extraer la vigencia real de cada póliza, asilando endosos de cancelación.

```
WITH Fecha_Corte AS (
  SELECT CURRENT_DATE AS fecha_evaluacion
),
Estado_Vigencia AS (
  SELECT p.id_poliza, p.fecha_inicio, p.fecha_fin AS fin_teorica,
  COALESCE(
    MAX(CASE WHEN e.tipo_endoso = 'Cancelacion'
    THEN e.fecha_efecto END), p.fecha_fin
  ) AS fecha_fin_real
  FROM polizas p
  LEFT JOIN endosos e ON p.id_poliza = e.id_poliza
  GROUP BY p.id_poliza, p.fecha_inicio, p.fecha_fin
)
```

***Eficiencia Relacional:** El uso de **COALESCE** y condicionales **CASE** en línea permite evitar múltiples subconsultas (*Self-Joins*) para extraer la fecha definitiva de corte.*

4.3 Paso 3: Evaluación Analítica mediante Window Functions

El cálculo de la prima histórica requiere iterar el estado financiero de cada transacción. Para evitar subconsultas correlacionadas en la cláusula **WHERE** (que colapsan la memoria RAM de Positron), se emplea **ROW_NUMBER()** de manera inversa.

```
Eventos_Financieros AS (  
  SELECT id_poliza, fecha_inicio AS fecha, prima_emitida AS mov  
  FROM polizas UNION ALL  
  SELECT id_poliza, fecha_efecto AS fecha, prima_adicional AS mov  
  FROM endosos WHERE tipo_endoso != 'Cancelacion'  
) ,  
Calculo_Exposicion AS (  
  SELECT ef.id_poliza, ef.fecha, ef.mov,  
  SUM(ef.mov) OVER (  
    PARTITION BY ef.id_poliza ORDER BY ef.fecha  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
  ) AS prima_acumulada,  
  ROW_NUMBER() OVER (  
    PARTITION BY ef.id_poliza ORDER BY ef.fecha DESC  
  ) AS orden_inverso  
  FROM Eventos_Financieros ef  
)
```

4.4 Paso 4: Cruce y Consolidación Actuarial

El ensamblaje final realiza uniones lógicas estrictas y cast de tipos para permitir cálculos decimales de fracción de tiempo de exposición.

```
SELECT ev.id_poliza, ev.fecha_inicio, ev.fecha_fin_real,  
ce.prima_acumulada AS prima_suscrita,  
ROUND(  
  ce.prima_acumulada *  
  (GREATEST(0, LEAST((SELECT fecha_evaluacion FROM Fecha_Corte)  
- ev.fecha_inicio, ev.fecha_fin_real - ev.fecha_inicio))::NUMERIC /  
  (ev.fin_teorica - ev.fecha_inicio)::NUMERIC), 2  
) AS prima_devengada  
FROM Estado_Vigencia ev  
LEFT JOIN Calculo_Exposicion ce  
ON ev.id_poliza = ce.id_poliza AND ce.orden_inverso = 1  
LEFT JOIN Siniestros_Reconciliados sr  
ON ev.id_poliza = sr.id_poliza;
```

5. Métricas de Optimización Computacional

El diseño del código evidencia un conocimiento avanzado de optimización de motores de bases de datos. Inicialmente, al usar una subconsulta `MAX(fecha)` dentro del condicional del `SELECT` principal, el planificador ejecutaba un *Nested Loop Scan* que intentaba procesar más de 1.15×10^{12} operaciones, extendiendo el tiempo de devolución a más de 6 minutos.

Al implementar `ROW_NUMBER() = 1`, la complejidad asintótica se redujo a $O(N \log N)$.

Método Estructural	Algoritmo en Planificador	Tiempo de Ejecución
Correlación de Subconsultas	<i>Nested Loop Scan</i>	> 360,000 ms
Optimización Window Function	<i>Hash Join</i>	~ 4,500 ms

Conclusión Técnica: Esta optimización demuestra la competencia analítica para manejar *Big Data* transaccional, requisito común en las áreas técnicas y de reservas de grandes corporativos aseguradores.

6. Referencias

PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation: Window Functions and Query Planning*.

Werner, M. (2018). *Data Analysis Using SQL and Excel*. John Wiley & Sons.

Society of Actuaries (SOA). (2020). *Introduction to Ratemaking and Loss Reserving for Property and Casualty Insurance*.